Vector Neurons: A General Framework for SO(3)-Equivariant Networks

March 24, 2021

Abstract

Invariance and equivariance to rigid transformations have been widely discussed in the 3D deep learning community. Yet most proposed methods use sophisticated mathematical tools limiting their accessibility, or rely on specific input data types or network architectures limiting their domain of use. In this paper, we introduce a general framework built on top of what we call Vector Neurons for creating SO(3) equivariant neural networks. Extending neurons from single scalars to 3D vectors, our vector neurons transport SO(3) actions to latent spaces and provide a framework for building equivariance in common neural operations including linear layers, non-linearities, pooling, and normalization. Because of their simplicity, vector neurons are very versatile and, as we demonstrate, can be incorporated into a rich set of diverse network architecture backbones, allowing them to process geometry inputs in arbitrary poses. Despite its simplicity, our method performs comparably well in accuracy and generalization with other more complex and specialized state-of-the-art methods.

1. Introduction

With the proliferation of lower-cost depth sensors, learning on 3D data has seen rapid progress in recent years. Of particular interest are point cloud networks, such as PointNet [24] or ACNe [29] that fully respect the inherent set symmetry – that point sets are not ordered – by incorporating order-invariant and/or order-equivariant layers. Yet, there are other important symmetries that have been less perfectly addressed in the context of point cloud processing, with 3D rotations being a prime example. Consider a scenario where one scans an object using their lidar-equipped phone to retrieve similar objects. Clearly, the global object pose should not affect the query result. PointNet uses spatial transformer layers [14], which at best only attain approximate pose invariance.

To avoid an exhaustive data augmentation to all possible rotations and translations, there is a need for network layers that are equivariant to both order and SO(3) symmetries. Recently, two approaches have been introduced to tackle this setting: Tensor Field Networks [31] and SE(3)-Transformers [12]. While guaranteeing equivariance by construction, both frameworks involve an intricate formulation and are hard to incorporate into existing pipelines as they are restricted to convolutions and rely on relative positions of adjacent points.

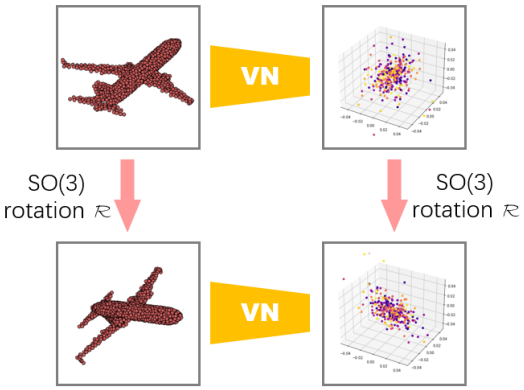

Figure 1: By lifting latent representations to 3-space we facilitate a simple rotation equivariant toolbox to create fully equivariant point cloud networks.

In this work, we address these issues by proposing a simple, lightweight framework to build SO(3) equivariant and invariant point cloud networks. A core ingredient in the proposed framework is our *Vector Neuron* (VN) unit, extending classical scalar processing neurons to vectors. While classical latent representations are vectors which can be thought of as ordered sequences of scalars, we instead deploy latent representations that are ordered sequences of 3-vectors (equivalently, $N \times 3$ matrices). Such a representation supports carrying rotations applied to an input point cloud to intermediate layers. However, this poses a challenge in the application of non-linear activations. In particular, standard point-wise activation functions such as

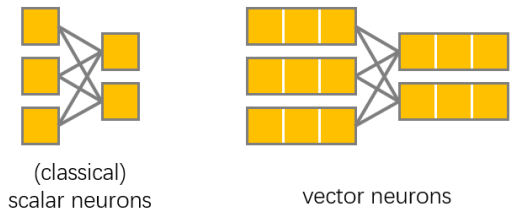

Figure 2: **Linear layer –** Typical neural networks today are built with "scalar" neurons – where the output of the non-linearities in a given layer is an ordered list of *scalars*. We extend deep networks to allow for "vector" neurons – where the output of the non-linearity is an ordered list of *vectors*.

ReLU will not commute with a rotation operation. A key contribution in this work is to vectorize the ReLU activation, lifting the non-linear activation function from 1D to 3D. When viewed as a vector, a classical ReLU activation simply zeros out the half-plane that points in its opposite direction. By dynamically predicting a data-dependent lifted activation direction in 3D, our vector-neuron layer becomes rotation equivariant. We further provide an invariant pooling operation as well as normalization layers, which renders our framework fully compatible with various point cloud network backbones. To demonstrate its versatility and efficiency, we implemented vector neuron versions of two popular architectures: PointNet, and DGCNN; and tested them on three different downstream tasks: classification (permutation invariant and rotation invariant), segmentation (permutation equivariant and rotation invariant), and reconstruction (rotation equivariant on the encoder side, and rotation invariant on the decoder side). Despite its simplicity and lightweight architecture, in all tasks, our VN achieved top performance when tested on randomly rotated shapes when compared with other equivariant architectures, and markedly improved performance compared to augmentation-induced equivariance.

To summarize, our key contributions are:

- We propose a new versatile framework for constructing SO(3)-equivariant pointcloud networks.
- Our building blocks are lightweight in terms of the number of learnable parameters and can be easily incorporated into multiple network architectures and
- We support a variety of learning tasks, in particular, we are the first to demonstrate a 3D equivariant network for 3D reconstruction.
- When evaluated on multiple tasks, our VN version of popular non-equivariant architectures achieves state-of-the-art performance.

Finally, though this paper is focused on 3D vector neurons, we remark that out vector neuron formulation naturally extends to n dimensions for any n, and provides equivariance to uniform scaling.

2. Related Work

The lack of robustness to rotation of classical deep learning architectures for point-cloud processing like PointNet [24], PointNet++ [25], Dynamic Graph CNN (DGCNN) [34], PCNN [2], PointCNN [19] (and many others) has driven interest for rotation invariant and equivariant designs. In recent years the field of rotation invariant and equivariant deep learning for geometry processing is rapidly developing. In what follows, we briefly review methods that achieve invariance and equivariance, as well as overview those that achieve equivariance via pose estimation.

Rotation invariant methods. Rotation invariance is a desirable property for tasks like shape classification or segmentation. Many rotation invariant architectures [20, 23, 6, 38, 40, 17, 41, 26] have been proposed to address these issues. For example, [6, 38, 40, 17] introduce cleverly designed rotation invariant operations. GC-Conv [40] relies on multi-scale reference frames based on PCA. RI-Framework [17] and LGR-Net [41] pairs local invariant information with global context. Some works like LGR-Net [41] use surface normals in addition to the points coordinates. SFCNN [26] propose an approach similar to multi-view by mapping input point-clouds to a sphere and performing operations on the sphere. Other works like [20, 23] rely on more principled approaches borrowing tools from equivariant deep learning.

Rotation equivariant methods. Recently multiple rotation equivariant deep learning architectures have emerged. A whole body of work is built on the theory of $\mathrm{SO}(3)$ representations [31, 15, 10, 35, 1], most of these works rely on the concept of convolution with steerable kernels bases. A steerable kernel basis is a family of function undergoing a rotation in function space given a rotation of their input parameter. Features computed through these convolution inherit this equivariant behavior. A rotation of the object in euclidean space induces a rotation of the features in feature space. We refer to [16] for a general theory of steerable kernels. Other works like EMVnet [11] consider a multi-view image based representation of the shapes based on renderings of meshes. In the context of pointcloud network, the universality of rotation equivariance has been studied in [9].

Equivariance via pose estimation. Qi et al. [24] achieved approximate pose equivariance by factoring out SO(3) transformations through object pose estimation. Most works in the literature study *instance-level* pose estimation, where the ground-truth canonical pose of the 3D CAD models corresponding to the input point cloud is available [4]. More recently Wang et al. [33] introduce *category-level* pose estimation, and extension to articulated objects has

also been proposed [18]. While both these methods [33, 18] need explicit 2D-to-3D supervision, relaxing supervision is possible by borrowing ideas from Transforming Auto-Encoders [13, 27]. However, while Sun et al. [30] learnt *category-level* as well as *multi-category* pose estimation in a fully unsupervised fashion, the underlying equivariant backbone [29] is only equivariant by *augmentation*.

3. Method

We introduce *Vector Neuron Networks* (VNNs), a straightforward extension to classical ReLU networks that provides $\mathrm{SO}(3)$ equivariance by construction. Neurons in standard artificial neural networks are represented as scalars $z \in \mathbb{R}$. When stacked into an ordered list, these neurons form a $C^{(d)}$ dimensional latent feature $\boldsymbol{z}=[z_1, z_2, \cdots, z_{C^{(d)}}]^\top \in \mathbb{R}^{C^{(d)}}$, where (d) is the layer depth[1].

However, when processing data embedded in $\mathbb{R}^3$ like 3D point clouds, structuring the effect of SO(3) transformations applied to the input shape on these vector hidden layers is not obvious. In particular, here we are interested in constructing rotation-equivariant learnable layers, namely layers that commute with the action of the rotation group.

To this end, we propose to "lift" the neuron representation from a scalar $z \in \mathbb{R}$ to a vector $\boldsymbol{v} \in \mathbb{R}^3$, which we dub Vector Neuron (VN). This results in list of Vector Neurons (matrix) $\boldsymbol{V}=[\boldsymbol{v}_1, \boldsymbol{v}_2, \cdots, \boldsymbol{v}_C]^\top \in \mathbb{R}^{C\times 3}$. Similar to standard latent representations, this vector-list feature can be used to encode an entire 3D shape, part of it, or a single point. In particular, when representing an (order-less) set of N points $\mathcal{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N\} \in \mathbb{R}^{N\times 3}$ in a point cloud we can consider a collection of N such vector-list features $\mathcal{V} = \{\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots, \boldsymbol{V}_N\} \in \mathbb{R}^{N\times C\times 3}$. Similar to standard neural networks, the number of latent channels $C^{(d)}$ can change between layers via a mapping:

$$\mathcal{V}^{(d+1)} = f(\mathcal{V}^{(d)}; \theta) : \mathbb{R}^{N\times C^{(d)}\times 3} \rightarrow \mathbb{R}^{N\times C^{(d+1)}\times 3}, \quad (1)$$

where θ represents learnable parameters.

Using this representation, the action of a rotation is made simple to implement, as it involves the application of a standard rotation matrix. Critically, we require that the mapping satisfies rotation equivariance, namely, for any rotation matrix $\boldsymbol{R} \in \mathrm{SO}(3)$:

$$f(\mathcal{V}R; \theta) = f(\mathcal{V}; \theta)R, \quad (2)$$

where we interpret the application of the rotation matrix to the set as $\mathcal{V}R = \{\boldsymbol{V}_i R\}_{i=1}^N$. To facilitate equivariance in standard point cloud network architectures, we construct

[1]For ease of notation, in what follows we will remove the layer index d whenever it is clear from context keeping in mind that the operations we introduce are per-layer.

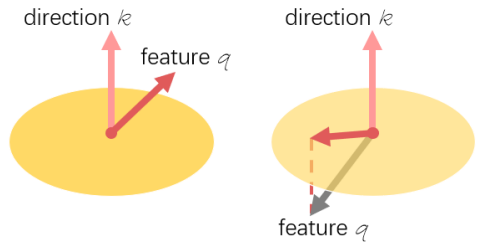

Figure 3: **Non-linearity –** Our non-linearity generalizes ReLU by acting on vectors rather than scalar inputs, and is parametric with respect to a direction $\boldsymbol{k}$: (left) when the input feature $\boldsymbol{q}$ lies in the half-space defined by $\boldsymbol{k}$, the feature stays unchanged; (right) when the input feature $\boldsymbol{q}$ lies in the half-space defined by $-\boldsymbol{k}$, the feature component in that half-space is clipped. See equation 6

VN layers following traditional designs via a combination of a linear map (Section 3.1) followed by a per-neuron non-linearity (Section 3.2). We additionally introduce equivariant pooling (Section 3.3) and normalization layers (Section 3.4). With these building blocks we are allowed to assemble a rich variety of complex neural networks in equivariance, including the most basic VN Multi-Layer Perceptron (VN-MLP) as a sequence of alternating linear and non-linear layers.

3.1. Linear layers – Figure 2

We begin by realizing the mapping f introduced in equation 1 as a linear operator – a fundamental module of neural networks. Given a weight matrix $\mathbf{W} \in \mathbb{R}^{C'\times C}$, we define a linear operation $f_{\text{lin}}(\cdot; \mathbf{W})$ acting on a vector-list feature $\boldsymbol{V} \in \mathcal{V} \in \mathbb{R}^{N\times C\times 3}$ as follows:

$$\boldsymbol{V}' = f_{\text{lin}}(\boldsymbol{V}; \mathbf{W}) = \mathbf{W}\boldsymbol{V} \in \mathbb{R}^{C'\times 3}. \quad (3)$$

We verify that a rotation matrix $R \in \mathrm{SO}(3)$ commutes with this linear layer:

$$f_{\text{lin}}(\boldsymbol{V}R; \mathbf{W}) = \mathbf{W}\boldsymbol{V}R = f_{\text{lin}}(\boldsymbol{V}; \mathbf{W})R = \boldsymbol{V}'R, \quad (4)$$

yielding the desired equivariance property. Note that we omit a bias term as an addition of a constant vector that would interfere with equivariance. Further, note that while this layer is SO(3) equivariant, we can achieve SE(3) equivariance by centering $\boldsymbol{V}$ at the origin. Finally, depending on the setting, $\mathbf{W}$ may or may not be the same for each element $\boldsymbol{V}$ of $\mathcal{V}$.

3.2. Non-linear layers – Figure 3 and Figure 4

Per-neuron non-linearity is key to the representation power of neural networks. As evident from recent literature, especially useful are functions that split the input domain into two half spaces and map them differently (e.g. ReLU,

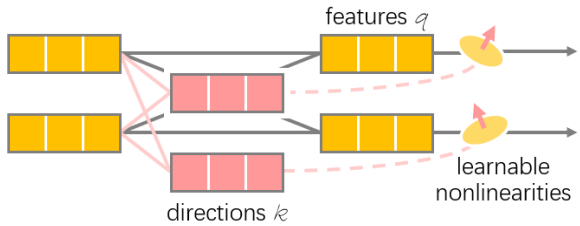

Figure 4: **Non-linear layer –** Our VN non-linearity is parametric in a direction $\boldsymbol{k}$, and we derive these parameters from the input features via a learnable linear layer.

leaky-ReLU, ELU, etc.). In the case of VN, a 3D version of these non-linearities, $\boldsymbol{V}' = f_{\text{ReLU}}(\boldsymbol{V})$, is needed. Yet, committing to a fixed frame (e.g. the standard coordinate system) would violate equivariance. Instead, we propose to dynamically predict a direction from the input vector-list feature. We then generalize the classical ReLU by projecting out the portion of a vector that points into the negative half-space of the learned direction.

More formally, given an input vector-list feature $\boldsymbol{V} \in \mathbb{R}^{C\times 3}$, for each output vector-neuron $\boldsymbol{v}' \in \boldsymbol{V}'$ we learn *two* weight matrices $\mathbf{W} \in \mathbb{R}^{1\times C}$ and $\mathbf{U} \in \mathbb{R}^{1\times C}$, linearly mapping the input feature $\boldsymbol{V}$ to a feature $\boldsymbol{q} \in \mathbb{R}^{1\times 3}$ and a direction $\boldsymbol{k} \in \mathbb{R}^{1\times 3}$:

$$\boldsymbol{q} = \mathbf{W}\boldsymbol{V}, \quad \boldsymbol{k} = \mathbf{U}\boldsymbol{V}\,. \tag{5}$$

We then define the output VN as:

$$\boldsymbol{v}' = \begin{cases} \boldsymbol{q} & \text{if } \langle \boldsymbol{q}, \boldsymbol{k}\rangle \geqslant 0 \\ \boldsymbol{q} - \left\langle \boldsymbol{q}, \frac{\boldsymbol{k}}{\|\boldsymbol{k}\|}\right\rangle \frac{\boldsymbol{k}}{\|\boldsymbol{k}\|} & \text{otherwise,} \end{cases} \tag{6}$$

resulting in an output vector-list: $f_{\text{ReLU}}(\boldsymbol{V}) = [\boldsymbol{v}']_{c=1}^{C}$.

As illustrated in Figure 3, $\boldsymbol{q}$ can be decomposed into two components: $\boldsymbol{q}_{\parallel}$ and $\boldsymbol{q}_{\perp}$ that are parallel and orthogonal to $\boldsymbol{k}$, respectively. Analogous to the standard scalar ReLU, we apply the nonlinear function to $\boldsymbol{q}_{\parallel}$ along the direction $\boldsymbol{k}$ by clipping $\boldsymbol{q}_{\parallel}$ to zero, while keeping $\boldsymbol{q}_{\perp}$ unchanged. Other types of split-case functions (e.g. leaky-ReLU) follow immediately from this definition. We discuss these and other types of non-linearities in the supplementary material.

It is easy to verify that f_{ReLU} is rotation equivariant. In particular, both $\boldsymbol{q}$ and $\boldsymbol{k}$ are linear maps of $\boldsymbol{V}$ and thus commute with a rotation matrix as discussed in (4). Moreover, the inner-product term in the second case would cancel out an orthogonal matrix $\langle \boldsymbol{q}\boldsymbol{R}, \boldsymbol{k}\boldsymbol{R}\rangle = \langle \boldsymbol{q}, \boldsymbol{k}\rangle$ resulting in a scalar multiplication of a $\boldsymbol{k}$, which is again equivariant.

3.3. Pooling layers

Pooling is widely used when aggregating local/global neighbourhood information, either spatially (e.g. PointNet++) or by feature similarity (e.g. DGCNN). While mean pooling is a linear operation that respects rotation equivariance, we also define a VN max pooling layer as a counterpart to the classical max pooling on scalars.

For global pooling, we are given a set of vector-lists $\mathcal{V} \in \mathbb{R}^{N\times C\times 3}$. We learn an element-wise signal of data dependent directions $\mathcal{K} \in \mathbb{R}^{N\times C\times 3}$. Similarly to Section 3.2, these directions are obtained via applying a weight matrix $\mathbf{W}_n \in \mathbb{R}^{C\times C}$ to each $\boldsymbol{V}_n \in \mathcal{V}$:

$$\mathcal{K} = \{\mathbf{W}_n \boldsymbol{V}_n\}_{n=1}^{N}, \tag{7}$$

and then computing the element of $\mathcal{V}$ that best aligns with $\mathcal{K}$ and selecting it as our global feature: $\forall$ channel $c \in [C]$,

$$f_{\text{MAX}}(\mathcal{V})_c = \boldsymbol{V}_{n^* c} \tag{8}$$

$$\text{where} \quad n^* = \arg\max_n \langle (\mathbf{W}_n \boldsymbol{V}_n)_c, \boldsymbol{V}_{nc}\rangle. \tag{9}$$

Similarly, we can aggregate information locally (local pooling) by grouping k nearest neighbours in $\mathcal{V}$ and perform the aforementioned pooling seperately for each group.

3.4. Normalization layers – Figure 5

Normalization often give rise to significant performance improvements. Layer [3] and instance normalizations [32] are done pre-sample (and the latter also per channel) and thus can be trivially generalized to VN networks, where the distributions are normalized with respect to vector distributions in $\mathbb{R}^3$.

Batch normalization. In contrast to other forms of normalizations, batch normalization aggregates statistics across all batch samples. While technically possible, in the context of rotation equivariant networks, averaging across arbitrarily rotated inputs would not necessarily be meaningful. For example, averaging two input features rotated in opposite

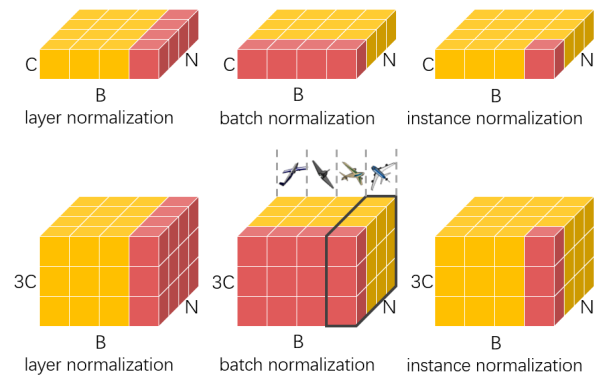

Figure 5: **Normalizations –** $\mathrm{B}, \mathrm{C}, \mathrm{N}$ stand for the batch, channel, and point dimensions respectively. (Top) classical scalar neurons; (bottom) vector neurons, batch normalization can only be done on vector norms as features within a batch are from different poses.

directions would zero them out instead of producing that feature in a canonical pose.

To tackle this issue, we instead apply batch normalization to the *invariant* component of the vector-list features, by normalizing the 2-norms of the vector-list features.

Given a batch of B vector-list features $\{\boldsymbol{V}_b\}_{b=1}^B$, our batch normalization is defined as:

$$\boldsymbol{N}_b = \text{ElementWiseNorm}(\boldsymbol{V}_b) \in \mathbb{R}^{N\times 1} \tag{10}$$

$$\{\boldsymbol{N}'_b\}_{b=1}^B = \text{BatchNorm}(\{\boldsymbol{N}_b\}_{b=1}^B) \tag{11}$$

$$\boldsymbol{V}'_b = \boldsymbol{V}_b \frac{\tilde{\boldsymbol{N}}_b}{\boldsymbol{N}_b}, \tag{12}$$

where $\text{ElementWiseNorm}(\boldsymbol{V}_b)$ computes the 2-norm of every vector channel $\boldsymbol{v}_c \in \boldsymbol{V}_b$.

3.5. Invariant layers

General invariant architectures are comprised of equivariant layers followed by invariant ones. We now introduce our invariant layer, that can be appended as needed to the output of the equivariant VN layers. Rotation-invariant networks are essential for both classification and segmentation tasks, where the identity of an object or its parts should be invariant to pose.

Key to our approach is the idea that the product of an equivariant signal $\boldsymbol{V} \in \mathbb{R}^{C\times 3}$ by the transpose of an equivariant signal $\boldsymbol{T} \in \mathbb{R}^{C'\times 3}$ is rotation invariant:

$$(\boldsymbol{V}R)(\boldsymbol{T}R)^\top = \boldsymbol{V}RR^\top\boldsymbol{T}^\top = \boldsymbol{V}\boldsymbol{T}^\top \tag{13}$$

Note that a specific case of (13) is the inner product of two equivariant vectors, in particular the norm of equivariant vector features is rotation invariant.

We could compute an invariant feature from a vector-list $\boldsymbol{V} \in \mathbb{R}^{C\times 3}$ as the Gram matrix $\boldsymbol{V}\boldsymbol{V}^\top$. However, this would result in a large $O(C^2)$ storage complexity. We could also consider taking the norm of each row of $\boldsymbol{V}$ but this would result in the loss of the relative directional information between the rows. Instead we propose a scalable solution with more manageable $O(C)$ complexity which can preserve directional information.

Our idea is to produce an equivariant coordinate system $\boldsymbol{T} \in \mathbb{R}^{3\times 3}$ from $\boldsymbol{V}$ and read $\boldsymbol{V}$ in this coordinate system thus producing rotation invariant features. In practice we consider our usual set of equivariant vector-list feature $\mathcal{V} \in \mathbb{R}^{N\times C\times 3}$. Inspired by Maron et al. [21], we produce a matrix $\boldsymbol{T}$ for each element by concatenating its feature $\boldsymbol{V} \in \mathbb{R}^{C\times 3}$ with the global mean $\overline{\boldsymbol{V}} := \frac{1}{N}\sum_n \boldsymbol{V}_n \in \mathbb{R}^{C\times 3}$ and running through f_{nonlin} with a target number of channel $C' = 3$.

$$\boldsymbol{T} := \text{VN-MLP}([\boldsymbol{V}, \overline{\boldsymbol{V}}]) \tag{14}$$

Finally we define our invariant layer by:

$$\text{VN-In}(\boldsymbol{V}_n) := \boldsymbol{V}_n\boldsymbol{T}_n^\top. \tag{15}$$

4. Network Architectures

We now show how we can plug vector neurons into two widely used 3D learning architectures, PointNet [24] and DGCNN [34]. These two backbones are representative of the richness of pointcloud networks, as PointNet is free from convolutions, and DGCNN comprises convolutions but the message passing is on dynamic graphs whose edges are not directly embedded in $\mathbb{R}^3$. As we show next, VN networks fit well into these backbones, while previous convolution-based methods such as TFN [31] and EGCL [28] do not. To make clear the ease of generalization, in the following we will adopt a naming convention to the functions defined in section 3 by using a "VN" prefix.

VN-DGCNN. DGCNN performs their permutation equivariant edge convolution by computing adjacent edge features $\boldsymbol{e}'_{nm}$ followed by a local max pooling:

$$\boldsymbol{e}'_{nm} = \text{ReLU}(\Theta(\boldsymbol{x}_m - \boldsymbol{x}_n) + \Phi\boldsymbol{x}_n) \tag{16}$$

$$\boldsymbol{x}'_n = \text{Pool}_{m:(n,m)\in\mathcal{E}}\ \boldsymbol{e}'_{nm} \tag{17}$$

where $\boldsymbol{x}_n \in \mathbb{R}^N$ are per-point features and Θ, Φ are learnable weight matrices. Our VN-DGCNN requires a straightforward modification:

$$\boldsymbol{E}'_{nm} = \text{VN-ReLU}(\Theta(\boldsymbol{V}_m - \boldsymbol{V}_n) + \Phi\boldsymbol{V}_n) \tag{18}$$

$$\boldsymbol{V}'_n = \text{VN-Pool}_{m:(n,m)\in\mathcal{E}}\ \boldsymbol{E}'_{nm} \tag{19}$$

using our vector-list representation $\boldsymbol{V}_n \in \mathbb{R}^{C\times 3}$.

VN-PointNet. PointNet approximates a permutation symmetric function using

$$\boldsymbol{x}' = \text{Pool}_{\boldsymbol{x}_n\in\mathcal{X}}(h(\boldsymbol{x}_1), \cdots, h(\boldsymbol{x}_N)) \tag{20}$$

where h is the same for all $\boldsymbol{x}_n$. In our VN version it writes as

$$\boldsymbol{V}' = \text{VN-Pool}_{\boldsymbol{V}_n\in\mathcal{V}}(f(\boldsymbol{V}_1), \cdots, f(\boldsymbol{V}_N)) \tag{21}$$

where f is a shared VN-MLP. One issue here exists in the first input layer where the input pointcloud coordinates $\boldsymbol{V}_i$ are $\mathbb{R}^{1\times 3}$ vectors and thus applying f to them would degenerate to a set of $\mathbb{R}^{C\times 3}$ vector-lists whose vector components are all linearly dependent (pointing to one direction). This is analogous to applying a per-pixel 1x1 convolution to a gray-scale image (single input channel). Therefore, in VN-PointNet we add an edge convolution at the input layer, mapping $\mathbb{R}^{1\times 3}$ features into $\mathbb{R}^{C\times 3}$ with $C > 1$ and then continue with per-point VN-MLP operations.

5. Experiments

We evaluate our method on three core tasks in pointcloud processing: classification (Section 5.1), segmentation (Section 5.2), and reconstruction (Section 5.3). In addition to their obvious diversity in the required output, these tasks

span different use cases of our proposed equivariant framework: classification and segmentation are rotation-invariant tasks, while reconstruction is rotation-equivariant.

Datasets. We employed the ModelNet40 [5] and the ShapeNet [5] datasets for evaluation. The ModelNet40 dataset consists of 40 classes with 12,311 CAD models in total. We used 9,843 models for training and the others for testing in the classification task. For the ShapeNet dataset, we followed [37] by using ShapeNet-part for part segmentation, which has 16 shape categories with more than 30,000 models. We also applied the subset of ShapeNet in [7] for shape reconstruction, containing 13 major categories with 50,000 models.

Train/test rotation setup. In classification and segmentation, following the conventions from Esteves et al. [10], we adopt three train/test settings: z/z, $z/\mathrm{SO}(3)$ and $\mathrm{SO}(3)/\mathrm{SO}(3)$, where z stands for data augmentation with rotations only around the z axis, and $\mathrm{SO}(3)$ for arbitrary rotations. All rotations are generated on the fly at train time, thereby comparing the equivariance-by-construction of VN architectures with a *augmentation-based* learned equivariances. At test time, each shape is presented at a single rotation. For reconstruction, we show results on extreme settings: no-rotation (I) – the standard evaluation setup for prior methods, and arbitrary rotations $\mathrm{SO}(3)$. Since outputs in this task are static and optimization for each shape takes multiple iterations at both train and test times, here the $\mathrm{SO}(3)$ random rotations are generated for each shape in a pre-processing stage and all shapes stay at fixed poses during training.

Network implementations. In classification and segmentation, we implement our VN networks in the identical architectures to their classical counterparts, but with each layer in the shape of $\lfloor \frac{N}{3} \rfloor \times 3$ while the corresponding layer in the scalar network has size N. This in fact greatly reduces the number of learnable parameters in VN networks, resulting in roughly $\leqslant 2/3^2 = 2/9$ times of parameters compared to the counterpart scalar networks – here the factor 2 in the numerator is because in nonlinearities two components $\boldsymbol{q}, \boldsymbol{k}$ are both learned (Equation 5). In reconstruction we slightly extend the layer size for the VN encoder. Moreover, in VN-PointNet, we discard the input spatial transformation MLP which learns 3×3 transformation matrices as our VN network already takes rigid transformations into consideration by construction. In the following experiments, we use mean pooling as aggregation in all networks, which performed better in practice. We will discuss more about the max pooling as well as ablation study on other structures in the supplementary material.

5.1. Classification – Table 1

We evaluate classification results on ModelNet40 compared with vanilla PointNet, DGCNN, and other rotation invariant or equivariant methods which takes point coordinates (meshes or point clouds) as inputs. Compared with their non-equivariant counterparts, the VN networks attain consistently good results on all the three settings, which indicates their robustness over rotations, especially in the $z/\mathrm{SO}(3)$ case where the test set contains unseen rotations in the train set. Even in the $\mathrm{SO}(3)/\mathrm{SO}(3)$ case with abundant train-time data augmentation, the rotation sensitive networks cannot perform as well as the equivariance by construction in VN networks. One the other hand, our VN network with DGCNN backbone (VN-DGCNN) outperforms all other equivariant or invariant methods with only point coordinate inputs in the $z/\mathrm{SO}(3)$ and $\mathrm{SO}(3)/\mathrm{SO}(3)$ cases. Note that methods that use surface normals [26, 41] still achieve better slightly better results.

Methods	z/z	$z/\mathrm{SO}(3)$	$\mathrm{SO}(3)/\mathrm{SO}(3)$
Point / mesh inputs			
PointNet [24]	85.9	19.6	74.7
DGCNN [34]	**90.3**	33.8	88.6
VN-PointNet	77.5	77.5	77.2
VN-DGCNN	89.5	**89.5**	**90.2**
PCNN [2]	92.3	11.9	85.1
ShellNet [39]	93.1	19.9	87.8
PointNet++ [25]	91.8	28.4	85.0
PointCNN [19]	92.5	41.2	84.5
Spherical-CNN [10]	88.9	76.7	86.9
a^3S-CNN [20]	89.6	87.9	88.7
SFCNN [26]	91.4	84.8	90.1
TFN [31]	88.5	85.3	87.6
RI-Conv [38]	86.5	86.4	86.4
SPHNet [23]	87.7	86.6	87.6
ClusterNet [6]	87.1	87.1	87.1
GC-Conv [40]	89.0	89.1	89.2
RI-Framework [17]	89.4	89.4	89.3
Point + normal inputs			
SFCNN [26]	92.3	85.3	91.0
LGR-Net [41]	90.9	90.9	91.1

Table 1: Test classification accuracy on the ModelNet40 dataset [36] in three train/test scenarios. z stands for aligned data augmented by random rotations around the vertical axis and $\mathrm{SO}(3)$ indicates data augmented by random rotations.

5.2. Part segmentation – Table 2

Table 2 shows our results in ShapeNet part segmentation. Again our method shows consistent results across different rotations and achieves best performance with VN-DGCNN compared with other works, even [41] which uses surface

Methods	z/SO(3)	SO(3)/SO(3)
Point / mesh inputs		
PointNet [24]	38.0	62.3
DGCNN [34]	49.3	78.6
VN-PointNet	72.4	72.8
VN-DGCNN	**81.4**	**81.4**
PointCNN [19]	34.7	71.4
PointNet++ [25]	48.3	76.7
ShellNet [39]	47.2	77.1
RI-Conv [38]	75.3	75.3
TFN [31]	76.8	76.2
GC-Conv [40]	77.2	77.3
RI-Framework [17]	79.2	79.4
Point + normal inputs		
LGR-Net [41]	80.0	80.1

Table 2: ShapeNet part segmentation. The results are reported in overall average category mean IoU over 16 categories in two train/test scenarios. With z, we refer to data augmented only by random rotations around the vertical axis, and SO(3) indicates random rotations.

normals in addition to the point coordinates.

5.3. Neural implicit reconstruction – Table 3

We follow the point cloud *completion* experiment from OccNet [22], where we reconstruct neural implicit functions from sparse and noisy input point clouds: we subsample 300 points from the surface of each (water-tight) ShapeNet model, and perturb them with normal noise with zero mean and $5e-3$ standard deviation. The outputs are occupancy probability functions $\mathcal{O}:\mathbb{R}^3\rightarrow[0,1]$ which can be parameterized by a shared neural implicit function $f_\theta(\cdot\mid\boldsymbol{z}):\mathbb{R}^3\rightarrow[0,1]$ conditioned by a latent code $\boldsymbol{z}$ derived from the input point set. For fair comparisons, we retrain the original OccNet [22] together with our methods for 300k iterations, and select the models with the best performance on the validation set.

Encoder network. We build an encoder-decoder framework with the architecture similar to [22] but in the language of VN. The encoder is rotation equivariant, encoding a point cloud $\{\boldsymbol{x}_1,\boldsymbol{x}_2,\cdots,\boldsymbol{x}_1\}$ into a global vector-list feature $\boldsymbol{Z}\in\mathbb{R}^{C\times 3}$. While in Mescheder et al. [22] the encoder is a PointNet, here we use a VN-PointNet:

$$\boldsymbol{Z}=\text{VN-PointNet}(\{\boldsymbol{x}_1,\boldsymbol{x}_2,\cdots,\boldsymbol{x}_1\}) \tag{22}$$

Decoder network. The decoder is rotation invariant between vector-list latent code $\boldsymbol{Z}\in\mathbb{R}^{C\times 3}$ and query point coordinate $\boldsymbol{x}\in\mathbb{R}^3$ – if you rotate the shape and the query point simultaneously the occupancy value stays unchanged. We define the decoder as a function on the three invariant features $\|\boldsymbol{x}\|^2,\langle\boldsymbol{x},\boldsymbol{Z}\rangle,\text{VN-In}(\boldsymbol{Z})$:

$$\mathcal{O}(\boldsymbol{x}\mid\boldsymbol{Z})=\text{VN-ResNet}([\langle\boldsymbol{x},\boldsymbol{Z}\rangle),\|\boldsymbol{x}\|^2,\text{VN-In}(\boldsymbol{Z})]) \tag{23}$$

where VN-In$(\cdot)$ is the VN invariant layer defined in Section 3.5. As an ablation study, we also replace VN-PointNet with a standard PointNet encoder (with the same invariant decoder), where the encoder generates latent codes $\boldsymbol{z}\in\mathbb{R}^C$ and we reshape them into $\boldsymbol{Z}\in\mathbb{R}^{(C/3)\times 3}$.

On the contrary, the decoder in [22] is a conditional batch norm (cbn) hyper-network $\mathcal{O}(\boldsymbol{x}\mid\boldsymbol{z})=f_{\text{cbn}}(\boldsymbol{x},\boldsymbol{z})$, which given latent code $\boldsymbol{z}\in\mathbb{R}^C$ and query point $\boldsymbol{x}\in\mathbb{R}^3$ outputs an occupancy probability $\mathcal{O}\in[0,1]$.

Quantitative results – Table 3. We evaluate the volumetric mean IoU of the reconstructions in the three train/test settings. Compared with the original OccNet, our equivariant-encoder/invariant-decoder exhibits excellent coherency in reconstructing shapes in any poses, with a minor loss in accuracy in the I/I case. Even simply adopting the invariant decoder without an equivariant encoder slightly improves the performances in all the three settings, which may be due to the $\mathbb{R}^3$ structure introduced to the latent codes.

Qualitative results – Figure 6. We show some reconstructions from the test set using the original OccNet (light pink) and our VN-OccNet (yellow). Meshes are extracted from the neural implicits using the Multi-resolution IsoSurface Extraction (MISE) method from Mescheder et al. [22]. Although OccNet better recognizes the fine details in the I/I case when the dataset is pre-aligned, it is extremely sensitive to rotations. In the I/SO(3) case when unseen rotations are applied at test time, OccNet totally fails by hardly learning anything meaningful; these findings are also consistent with those in Deng et al. [8]. Even in the SO(3)/SO(3) case when data augmentation is adopted at train time, it still shows its limitation by generating blurry shapes (top left), averaged shapes (top right, the box-like output consists of sofa features averaged from different poses), or shapes with incorrect priors (bottom right, a shape in the car class is falsely identified as a chair).

6. Conclusions

We have introduced Vector Neurons – a novel framework that facilitates rotation equivariant neural networks by lifting standard neural network representations to 3 space. To that end, we have introduced the vector-neuron counterpart of standard network modules: linear layers, non-linearities, pooling and normalization. Using our framework, we have built a rotation-equivariant version of two leading point-cloud network backbones: PointNet and DGCNN, and evaluated them on 3 tasks: classification, segmentation and reconstruction. Our results demonstrate a consistent advan-

Methods	Encoder	Latent code	Decoder	I/I	I/SO(3)	SO(3)/SO(3)
OccNet [22]	PointNet	$\boldsymbol{z} \in \mathbb{R}^C$	$f_{\text{cbn}}(\boldsymbol{x}, \boldsymbol{z})$	71.4	30.9	58.2
-	PointNet	$\boldsymbol{Z} \in \mathbb{R}^{\lfloor C/3 \rfloor \times 3}$	$f(\langle \boldsymbol{x}, \boldsymbol{Z} \rangle), \lVert \boldsymbol{x} \rVert^2, \text{VN-In}(\boldsymbol{Z}))$	**72.0**	31.4	58.9
VN-OccNet	VN-PointNet	$\boldsymbol{Z} \in \mathbb{R}^{\lfloor C/3 \rfloor \times 3}$	$f(\langle \boldsymbol{x}, \boldsymbol{Z} \rangle), \lVert \boldsymbol{x} \rVert^2, \text{VN-In}(\boldsymbol{Z}))$	67.1	**67.0**	**67.6**

Table 3: Volumetric mIoU on ShapeNet reconstruction with neural implicits.

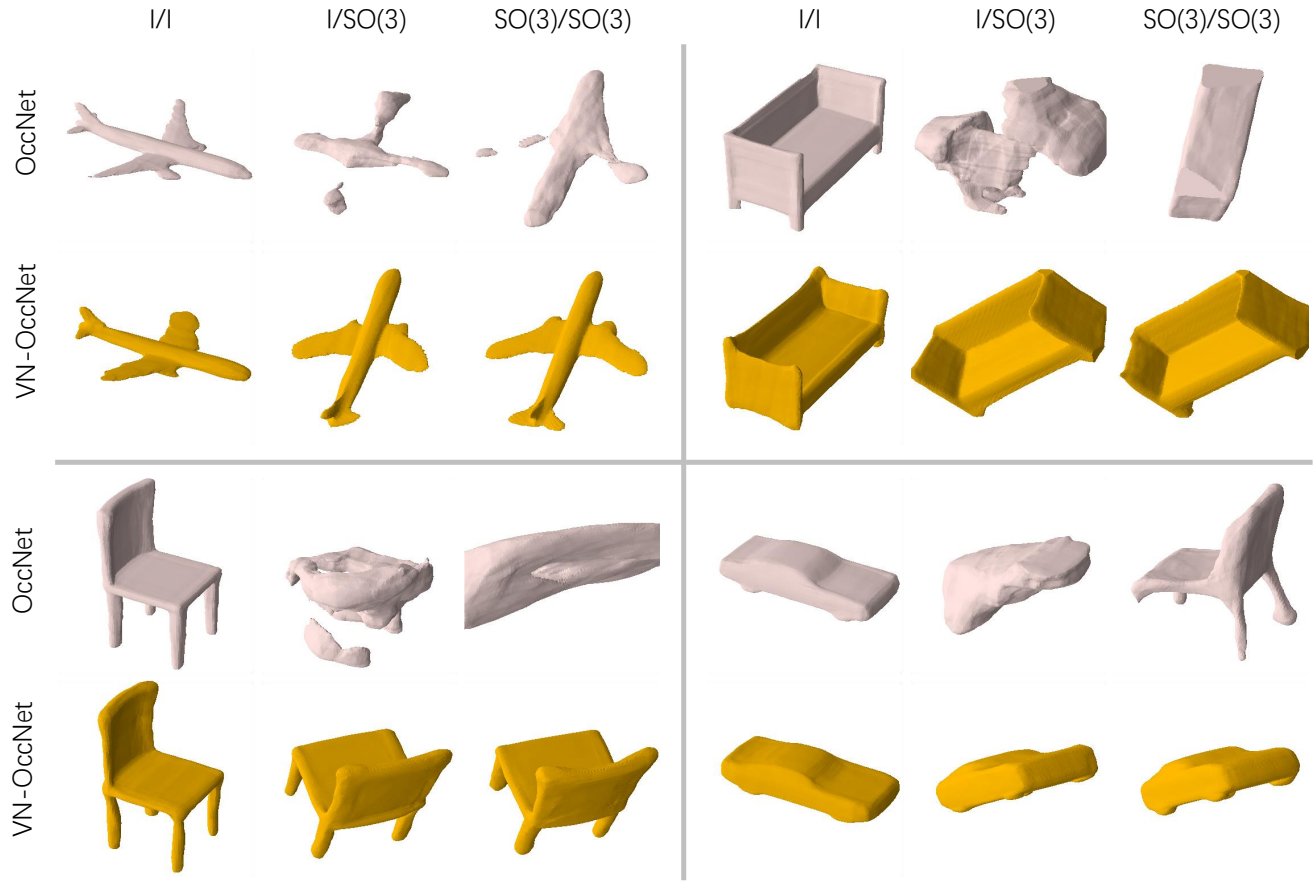

Figure 6: Reconstruction results with OccNet OccNet (light pink) and VN-OccNet (yellow).

tage to our modified architecture when the input shapes pose is arbitrary, compared to an augmentation based approach.

Limitation and future work. While our method shines under arbitrary rotation settings, on aligned input shapes and specifically in the task of reconstruction, our VN-OccNet was not able to match the reconstruction quality of vanilla OccNet by a small margin. In future work we plan to investigate this matter.

In this work, we have focused on point cloud networks, yielding permutation and rotation equivariant architectures, yet our framework is *not* limited to 3D inputs and can find applications in other modalities like meshes, voxel grids, and even in the image domain. Generalization of vector neurons to other transformation groups of interest, such as the affine group, can also be investigated.

In summary, by making rotation equivariant modules simple and accessible we hope to alleviate the need to curate and pre-align shapes for supervision and inspire future research on this fascinating topic.

References

[1] Brandon Anderson, Truong-Son Hy, and Risi Kondor. Cormorant: Covariant molecular neural networks. *arXiv preprint arXiv:1906.04015*, 2019. 2

[2] Matan Atzmon, Haggai Maron, and Yaron Lipman. Point convolutional neural networks by extension operators. *arXiv preprint arXiv:1803.10091*, 2018. 2, 6

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. 4

[4] Eric Brachmann, Alexander Krull, Frank Michel, Stefan Gumhold, Jamie Shotton, and Carsten Rother. Learning 6D object pose estimation using 3D object coordinates. In *European Conference on Computer Vision*, pages 536–551, 2014. 2

[5] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical Report arXiv:1512.03012 [cs.GR], 2015. 6

[6] Chao Chen, Guanbin Li, Ruijia Xu, Tianshui Chen, Meng Wang, and Liang Lin. Clusternet: Deep hierarchical cluster network with rigorously rotation-invariant representation for point cloud analysis. In *Conference on Computer Vision and Pattern Recognition*, pages 4994–5002, 2019. 2, 6

[7] Christopher B Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction. In *European Conference on Computer Vision*, pages 628–644, 2016. 6

[8] Boyang Deng, JP Lewis, Timothy Jeruzalski, Gerard Pons-Moll, Geoffrey Hinton, Mohammad Norouzi, and Andrea Tagliasacchi. Nasa: Neural articulated shape approximation, 2020. 7

[9] Nadav Dym and Haggai Maron. On the universality of rotation equivariant point cloud networks, 2020. 2

[10] Carlos Esteves, Christine Allen-Blanchette, Ameesh Makadia, and Kostas Daniilidis. Learning SO(3) equivariant representations with spherical cnns. In *European Conference on Computer Vision*, pages 52–68, 2018. 2, 6

[11] Carlos Esteves, Yinshuang Xu, Christine Allen-Blanchette, and Kostas Daniilidis. Equivariant multi-view networks. In *International Conference on Computer Vision*, pages 1568–1577, 2019. 2

[12] Fabian B Fuchs, Daniel E Worrall, Volker Fischer, and Max Welling. Se(3)-transformers: 3d roto-translation equivariant attention networks. *arXiv preprint arXiv:2006.10503*, 2020. 1

[13] Geoffrey E Hinton, Alex Krizhevsky, and Sida D Wang. Transforming auto-encoders. In *International conference on artificial neural networks*. Springer, 2011. 3

[14] Max Jaderberg, Karen Simonyan, Andrew Zisserman, et al. Spatial transformer networks. In *Advances in Neural Information Processing Systems*, pages 2017–2025, 2015. 1

[15] Risi Kondor, Zhen Lin, and Shubhendu Trivedi. Clebsch-gordan nets: a fully fourier space spherical convolutional neural network. *arXiv preprint arXiv:1806.09231*, 2018. 2

[16] Leon Lang and Maurice Weiler. A wigner-eckart theorem for group equivariant convolution kernels. *arXiv preprint arXiv:2010.10952*, 2020. 2

[17] Xianzhi Li, Ruihui Li, Guangyong Chen, Chi-Wing Fu, Daniel Cohen-Or, and Pheng-Ann Heng. A rotation-invariant framework for deep point cloud analysis. *arXiv preprint arXiv:2003.07238*, 2020. 2, 6, 7

[18] Xiaolong Li, He Wang, Li Yi, Leonidas J Guibas, A Lynn Abbott, and Shuran Song. Category-level articulated object pose estimation. In *Conference on Computer Vision and Pattern Recognition*, pages 3706–3715, 2020. 3

[19] Yangyan Li, Rui Bu, Mingchao Sun, Wei Wu, Xinhan Di, and Baoquan Chen. PointCNN: Convolution on x-transformed points. In *Advances in Neural Information Processing Systems*, pages 820–830, 2018. 2, 6, 7

[20] Min Liu, Fupin Yao, Chiho Choi, Ayan Sinha, and Karthik Ramani. Deep learning 3d shapes using alt-az anisotropic 2-sphere convolution. In *International Conference on Learning Representations*, 2018. 2, 6

[21] Haggai Maron, Or Litany, Gal Chechik, and Ethan Fetaya. On learning sets of symmetric elements. In *International Conference on Machine Learning*, pages 6734–6744, 2020. 5

[22] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3D reconstruction in function space. In *Conference on Computer Vision and Pattern Recognition*, pages 4460–4470, 2019. 7, 8

[23] Adrien Poulenard, Marie-Julie Rakotosaona, Yann Ponty, and Maks Ovsjanikov. Effective rotation-invariant point cnn with spherical harmonics kernels. In *IEEE International Conference on 3D Vision*, pages 47–56, 2019. 2, 6

[24] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. Pointnet: Deep learning on point sets for 3D classification and segmentation. In *Conference on Computer Vision and Pattern Recognition*, pages 652–660, 2017. 1, 2, 5, 6, 7

[25] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, pages 5099–5108, 2017. 2, 6, 7

[26] Yongming Rao, Jiwen Lu, and Jie Zhou. Spherical fractal convolutional neural networks for point cloud recognition. In *Conference on Computer Vision and Pattern Recognition*, pages 452–460, 2019. 2, 6

[27] Helge Rhodin, Mathieu Salzmann, and Pascal Fua. Unsupervised geometry-aware representation for 3d human pose estimation. In *European Conference on Computer Vision*, pages 750–767, 2018. 3

[28] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. E(n) equivariant graph neural networks. *arXiv preprint arXiv:2102.09844*, 2021. 5

[29] Weiwei Sun, Wei Jiang, Andrea Tagliasacchi, Eduard Trulls, and Kwang Moo Yi. Attentive context normalization for robust permutation-equivariant learning. In *Conference on Computer Vision and Pattern Recognition*, 2020. 1, 3

[30] Weiwei Sun, Andrea Tagliasacchi, Boyang Deng, Sara Sabour, Soroosh Yazdani, Geoffrey Hinton, and Kwang Moo Yi. Canonical capsules: Unsupervised capsules in canonical pose, 2020. 3

[31] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. Tensor field networks: Rotation-and translation-equivariant neural networks for 3D point clouds. *arXiv preprint arXiv:1802.08219*, 2018. 1, 2, 5, 6, 7

[32] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization, 2017. 4

[33] He Wang, Srinath Sridhar, Jingwei Huang, Julien Valentin, Shuran Song, and Leonidas J Guibas. Normalized object coordinate space for category-level 6D object pose and size estimation. In *Conference on Computer Vision and Pattern Recognition*, pages 2642–2651, 2019. 2, 3

[34] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph CNN for learning on point clouds. *ACM Transactions on Graphics*, 38(5):1–12, 2019. 2, 5, 6, 7, 11

[35] Maurice Weiler, Mario Geiger, Max Welling, Wouter Boomsma, and Taco Cohen. 3d steerable cnns: Learning rotationally equivariant features in volumetric data. *arXiv preprint arXiv:1807.02547*, 2018. 2

[36] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Conference on Computer Vision and Pattern Recognition*, pages 1912–1920, 2015. 6

[37] Li Yi, Vladimir G Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Cewu Lu, Qixing Huang, Alla Sheffer, and Leonidas Guibas. A scalable active framework for region annotation in 3d shape collections. *ACM Transactions on Graphics*, 35(6):1–12, 2016. 6

[38] Zhiyuan Zhang, Binh-Son Hua, David W Rosen, and Sai-Kit Yeung. Rotation invariant convolutions for 3d point clouds deep learning. In *IEEE International Conference on 3D Vision*, pages 204–213, 2019. 2, 6, 7

[39] Zhiyuan Zhang, Binh-Son Hua, and Sai-Kit Yeung. Shellnet: Efficient point cloud convolutional neural networks using concentric shells statistics. In *International Conference on Computer Vision*, pages 1607–1616, 2019. 6, 7

[40] Zhiyuan Zhang, Binh-Son Hua, Wei Chen, Yibin Tian, and Sai-Kit Yeung. Global context aware convolutions for 3d point cloud understanding. *arXiv preprint arXiv:2008.02986*, 2020. 2, 6, 7

[41] Chen Zhao, Jiaqi Yang, Xin Xiong, Angfan Zhu, Zhiguo Cao, and Xin Li. Rotation invariant point cloud classification: Where local geometry meets global topology. *arXiv preprint arXiv:1911.00195*, 2019. 2, 6, 7

Vector Neurons: A General Framework for SO(3)-Equivariant Networks

(Supplementary Material)

7. Discussions

In this section, we discuss some extensions, alternatives, and explanations to the VN layers in Section 3.

7.1. Non-linearity

Linear and non-linear layers. The VN-ReLU defined in Section 3.2 already consists of a build-in linear layer $\boldsymbol{q} = \mathbf{W}\boldsymbol{V}$ (Equation 5) and the the non-linearity is applied to this learned feature $\boldsymbol{q}$. An alternative to this is to construct linear and non-linear layers separately, where the non-linearity is directly applied to the input vector channel $\boldsymbol{v} \in \boldsymbol{V}$ by

$$\boldsymbol{v}' = \begin{cases} \boldsymbol{v} & \text{if } \langle \boldsymbol{v}, \boldsymbol{k} \rangle \geqslant 0 \\ \boldsymbol{v} - \left\langle \boldsymbol{v}, \frac{\boldsymbol{k}}{\|\boldsymbol{k}\|} \right\rangle \frac{\boldsymbol{k}}{\|\boldsymbol{k}\|} & \text{otherwise,} \end{cases} \tag{24}$$

Detaching the linear layer from non-linearity allows more flexibility in constructing neural networks and, in practice, gives better results in some cases. However, this also doubles the network depth and can result in much longer training time compared to the entangled linear-ReLU layer in Equation 6. Experimental comparisons will further be shown in Section 8.2.

Other non-linearities. Though we only showed how to define VN-ReLU in Section 3.2, an entire library of equivariant non-linearities can be defined in this manner using the input-dependent direction vector $\boldsymbol{k}$. An immediate extension is VN-LeakyReLU, where instead of clipping $\boldsymbol{q}_{\|}$ to zero we contract it by a factor $\alpha \in (0, 1)$. Following the detached VN-ReLU in 24, the VN-LeakyReLU can be easily expressed as:

$$f_{\text{LeakyReLU}}(\boldsymbol{V}; \alpha) = \alpha \boldsymbol{V} + (1 - \alpha) f_{\text{ReLU}}(\boldsymbol{V}). \tag{25}$$

An entangled layer of VN-Linear and VN-LeakyReLU can also be defined analogous to Equation 6. More generally, given an arbitrary non-linear scalar function $f : \mathbb{R} \to \mathbb{R}$, we can incorporate it into our VN non-linearity framework by applying it to $\boldsymbol{q}_{\|}$ along the $\boldsymbol{k}$ direction, namely,

$$\boldsymbol{v}' = \frac{h(\|\boldsymbol{q}_{\|}\|)}{\|\boldsymbol{q}_{\|}\|} \boldsymbol{q}_{\|} + \boldsymbol{q}_{\perp}. \tag{26}$$

7.2. Local Pooling

The VN-MAX pooling in Section 3.3 is defined across an entire point cloud $\mathcal{V} \in \mathbb{R}^{N \times C \times 3}$, but we can also aggregate information locally via local pooling.

In the primal space. For any point $\boldsymbol{x}_n \in \mathcal{X}$ with feature $\boldsymbol{V}_n \in \mathcal{V}$ we consider its K nearest neighbours $\{\boldsymbol{x}_k\}_{k=1}^K$ in the primal space and we denote by $\boldsymbol{V}_k$ the corresponding feature of $\boldsymbol{x}_k$ in $\mathcal{V}$. Similar to the global pooling (Equation 9), the local pooling (in the primal space) is given by:

$$f_{\text{MAX}}(\boldsymbol{V}_n, \{\boldsymbol{V}_k\}_{k=1}^K)_c = \boldsymbol{V}_{k^* c} \tag{27}$$

$$\text{where} \quad k^* = \arg\max_k \langle (\mathbf{W}_n \boldsymbol{V}_n)_c, \boldsymbol{V}_{nc} \rangle \tag{28}$$

In the feature space. As in DGCNN [34], we can also query the K nearest neighbours $\{\boldsymbol{K}_k\}_{k=1}^K$ of feature $\boldsymbol{V}_n \in \mathcal{V}$ in the feature space $\mathbb{R}^{C \times 3}$ directly, followed by the local pooling (in the feature space) in exactly the same formulation as Equation 28.

7.3. Batch Normalization

In VN-BatchNorm (Equation 10), for each input vector-list feature $\boldsymbol{V}_b$, all entries in its per-channel 2-norm $\boldsymbol{N}_b$ are non-negative, but after normalizing the distributions, the output "2-norms" $\boldsymbol{N}'_b$ can have negative entries. Geometrically, a negative entry $\boldsymbol{n}'_{bc} \in \boldsymbol{N}'_b$ means flipping the orientation of its corresponding vector channel, namely, $\boldsymbol{v}'_c \in \boldsymbol{V}'_b$ is in the opposite direction of $\boldsymbol{v}_c \in \boldsymbol{V}_b$.

8. Additional Experiments

8.1. Training on Aligned Data

In Section 5, we adopt the three train/test settings z/z, $z/\text{SO}(3)$, $\text{SO}(3)/\text{SO}(3)$ from prior works to standardize the comparisons between different methods. However, it is also interesting to see how each method performs when trained trained without augmentation (no-rotation setting I) but tested on rotated shapes. Table 4 shows our additional classification and part segmentation results on I/I, I/z, and I/SO(3). Compared to the z-trained settings in Table 1 and Table 2, here we further highlight the robustness of our VN networks on test-time rotations in contrast to their rotation-sensitive counterparts.

8.2. Ablation Study

In this section, we use the ModelNet40 classification task to show multiple ablation studies on VN network structures. All experimental settings here are exactly the same as in Section 5.1 Table 1.

Non-linearity. We compare the performances of an entangled linear-ReLU (or linear-LeakyReLU) layer in Equation 6 with a 2-tuple of a linear layer plus a separate non-linearity in Equation 24. Table 5 shows how these two structures performs when plugged into VN-PointNet and VN-DGCNN backbones. Within both backbones, separating

Methods	Classification (%)			Segmentation (mIoU)		
	I/I	I/z	I/SO(3)	I/I	I/z	I/SO(3)
PointNet						
DGCNN	**92.9**	37.2	16.6	**85.2**	43.8	36.1
VN-PointNet						
VN-DGCNN	90.0	**90.0**	**90.0**	81.5	**81.5**	**81.5**

Table 4

linear and non-linear layers leads to slightly better results. But this is also at the cost of a much longer training time (roughly $\geqslant 1.5$ times to the entangled versions).

Mean and Max Pooling. Table 6 shows the comparisons of mean and VN-MAX aggregations. VN-MAX introduces additional learnable weights than the mean aggregation.

Invariance. Table 7 shows our ablation study on the invariant layer VN-In in Section 3.5. Specifically, in computing equivariant coordinate system $\boldsymbol{T}$ following Equation 13, we compare the combinations of the following options: whether or not concatenating the global mean $\overline{\boldsymbol{V}}$ to the local feature $\boldsymbol{V}$, and whether the VN-MLP is a 3-layer MLP or a single linear layer. Both the global mean $\overline{\boldsymbol{V}}$ the 3-layer VN-MLP help improving the performance.

9. The reconstruction equivariant decoder

TODO.... DINNER TIME.

Method	Non-linearity	z/z	$z/\mathrm{SO}(3)$	$\mathrm{SO}(3)/\mathrm{SO}(3)$
VN-PointNet	VN-linear + $\boldsymbol{k}$-ReLU			
VN-PointNet	$(\boldsymbol{q}, \boldsymbol{k})$-ReLU			
VN-DGCNN	VN-linear + $\boldsymbol{k}$-LeakyReLU			
VN-DGCNN	$(\boldsymbol{q}, \boldsymbol{k})$-LeakyReLU			

Table 5

Methods	Pooling	z/z	$z/\mathrm{SO}(3)$	$\mathrm{SO}(3)/\mathrm{SO}(3)$
VN-PointNet	MEAM			
VN-PointNet	VN-MAX			
VN-DGCNN	MEAN			
VN-DGCNN	VN-MAX			

Table 6

Methods	VN-In	z/z	$z/\mathrm{SO}(3)$	$\mathrm{SO}(3)/\mathrm{SO}(3)$
VN-PointNet	VN-linear			
VN-PointNet	VN-linear + $\overline{\boldsymbol{V}}$			
VN-PointNet	VN-MLP			
VN-PointNet	VN-MLP + $\overline{\boldsymbol{V}}$			
VN-DGCNN	VN-linear			
VN-DGCNN	VN-linear + $\overline{\boldsymbol{V}}$			
VN-DGCNN	VN-MLP			
VN-DGCNN	VN-MLP + $\overline{\boldsymbol{V}}$			

Table 7